

HOMEWORK 6 · FINAL CAPSTONE

AI-Powered Transaction Processing Pipeline

Agent microservices over REST, driven by configuration —
with a React dashboard, a custom MCP server, and a coverage gate.

Created by Mykola Bernadskyi

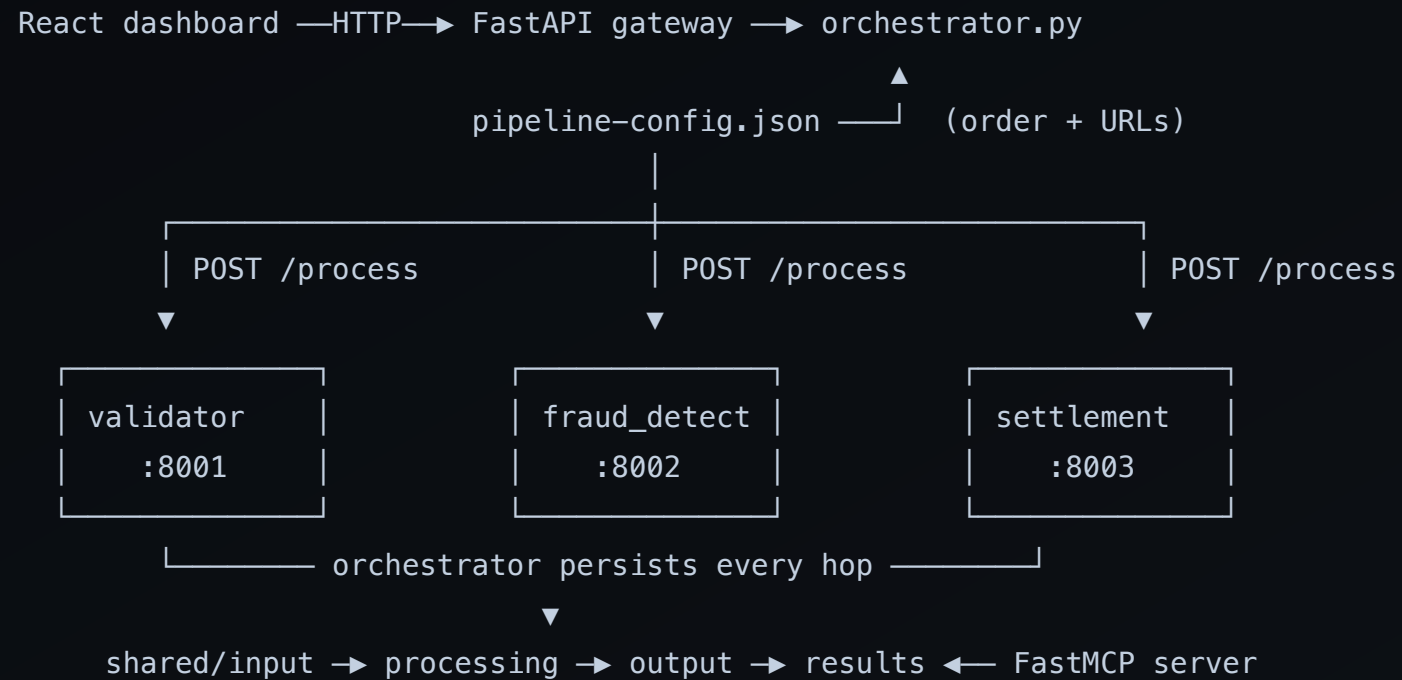
OVERVIEW

What it does

- Processes bank transactions through **three independent agent microservices**
- **Validation** → **Fraud Detection** → **Settlement**, each its own FastAPI service
- The orchestrator calls them over **HTTP REST**, in an order read from **configuration**
- Every hop is persisted to **shared/** as a JSON envelope — a full audit trail
- Money is **decimal.Decimal** everywhere; account numbers are PII and never logged
- Observed via a **React dashboard**; queryable via a custom **MCP server**

ARCHITECTURE

Microservices + config-driven orchestration



Agents never call each other — only the orchestrator calls them

THE KEY IDEA

Execution order is **data**, not code

```
{
  "agents": [
    { "name": "validator",
      "url": "http://localhost:8001/process" },
    { "name": "fraud_detector",
      "url": "http://localhost:8002/process" },
    { "name": "settlement",
      "url": "http://localhost:8003/process" }
  ],
  "timeout_seconds": 10
}
```

- ▶ The orchestrator **reads this file** and calls each service in the listed order
- ▶ Reorder, add, or repoint an agent — **zero code changes**
- ▶ Each agent is a thin HTTP wrapper around a **pure function**
- ▶ Unreachable service → a clear **AgentUnreachableError**

Three agents, one interface

```
POST /process { ...transaction record... } → { ...updated record... }  
GET  /health  → { "status": "ok", "agent": "validator" }
```

- ▶ **Validator** — required fields, Decimal amount (negative only for refunds), ISO 4217 currency; rejects with a reason
- ▶ **Fraud Detector** — additive risk 0.0–1.0: high-value +0.5, off-hours +0.3, cross-border +0.2, wire +0.1; flags ≥ 0.5
- ▶ **Settlement** — 0.5% fee + net with ROUND_HALF_UP; preserves the fraud flag; rejected never settle

Outcomes

8

Total processed

4

Settled

3

Flagged

1

Rejected

- ▶ **TXN002 / TXN005** — \$25k / \$75k wires → flagged (high-value + wire)
- ▶ **TXN004** — 02:47 UTC, Germany → flagged (off-hours + cross-border)
- ▶ **TXN006** — currency "XYZ" → rejected (not ISO 4217)
- ▶ **TXN007** — -\$100 refund → allowed (negative permitted for refunds)

Stack & quality gate

Agents	FastAPI microservices
Transport	HTTP REST (httpx)
Money	decimal.Decimal
Frontend	React + TS + Vite
MCP server	FastMCP 3
Tests	pytest + pytest-cov

98%

Coverage · 67 tests · gate floor 80%

- ▶ Coverage gate **blocks push** below 80%
- ▶ git pre-push + Claude Code hook
- ▶ Tests mock HTTP — **hermetic**, no live services

REFLECTION

Lessons learned

- ▶ **Spec first.** Writing the specification before code made every agent boundary obvious
- ▶ **Topology belongs in config.** Moving the agent order out of code made the pipeline reconfigurable
- ▶ **Files are the persistence.** agents.md + shared/ let any session — or service — resume cleanly
- ▶ **context7 beat memory.** Live docs for Decimal & CORS avoided stale-API guesswork
- ▶ **The hook is the point.** A machine-enforced coverage gate outlives good intentions
- ▶ **Determinism matters** in finance — the same transaction must never be judged twice differently

Thank you — Mykola Bernadskyi